Comp90042 Natural Language Processing

Workshop (Week5 | 2026s1)

Dr Jean Lee

Agenda

1. Discussion (~40 mins)
   - **Neural Network (NN) & NN Language Models**
   - **Feed Forward NN (FFNN), Recurrent NN (RNN)**
   - **Matrix Calculation**
2. Programming (~20 mins)
   - **PyTorch Implementation for Deep Learning**

30th Mar	5	workshop-05.pdf	07-deep-learning-pytorch.ipynb 07-yelp-dataset.txt	*Release on end of 3rd April

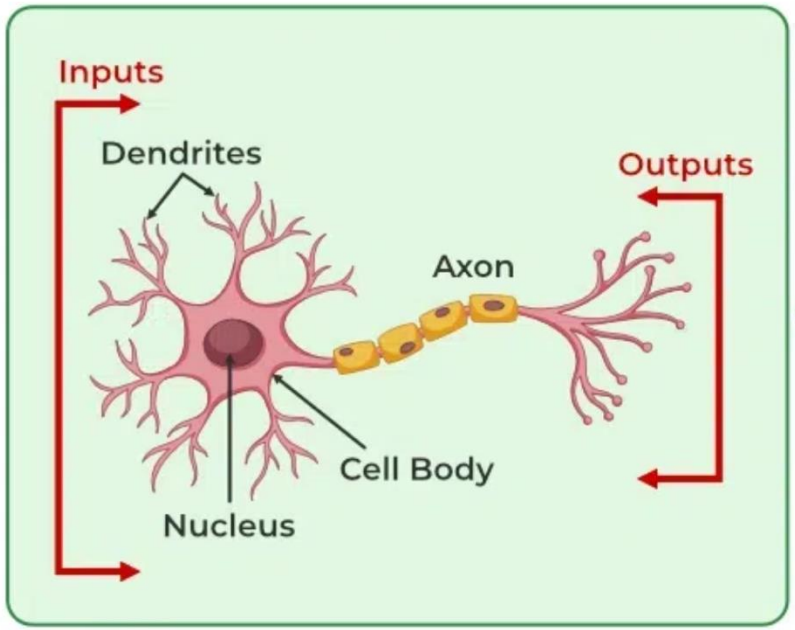
Discussion : Neural Network Language Model

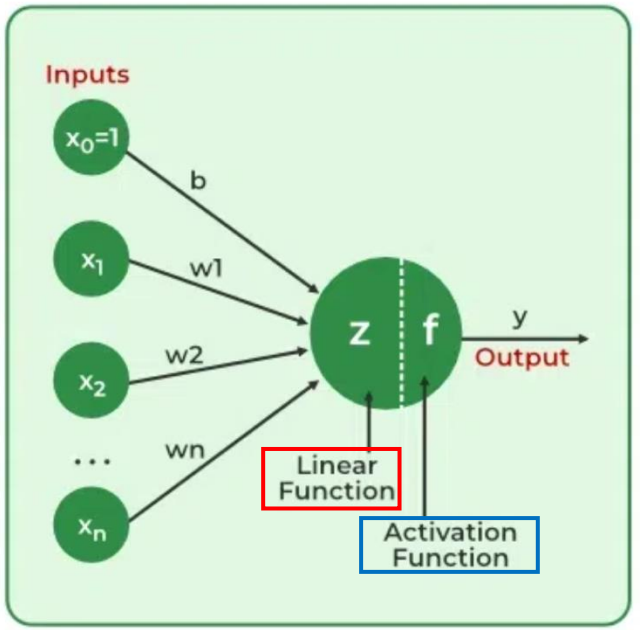
Discussion

1. How does a neural network language model (feedforward or recurrent) handle a large vocabulary, and how does it deal with sparsity (i.e. unseen sequences of words)?
2. Why do we say most parameters of a neural network language model (feedforward or recurrent) is in their input and output word embeddings?
3. What advantage does an RNN language model have over N-gram language model?
4. What is the vanishing gradient problem in RNN, and what causes it? How do we tackle vanishing gradient for RNN?

What is Neural Networks (NN)?

- ML/DL models that mimic the complex functions of the human brain
- First mathematical model for Artificial Neurons (1940-1950)
- **Perceptron** (1960-1970) : One-layer Artificial Neural Network





Weighted sum

$$h = \tanh\left(\sum_j w_j x_j + b\right)$$

Non-linear transform

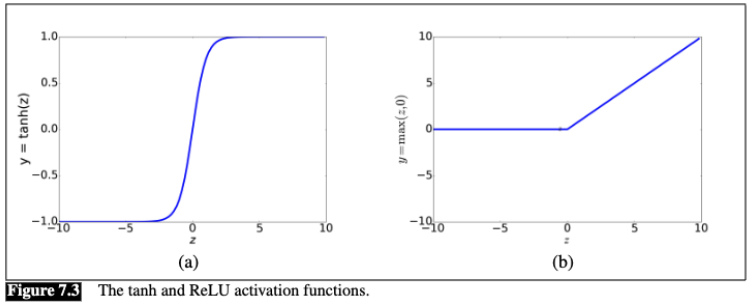
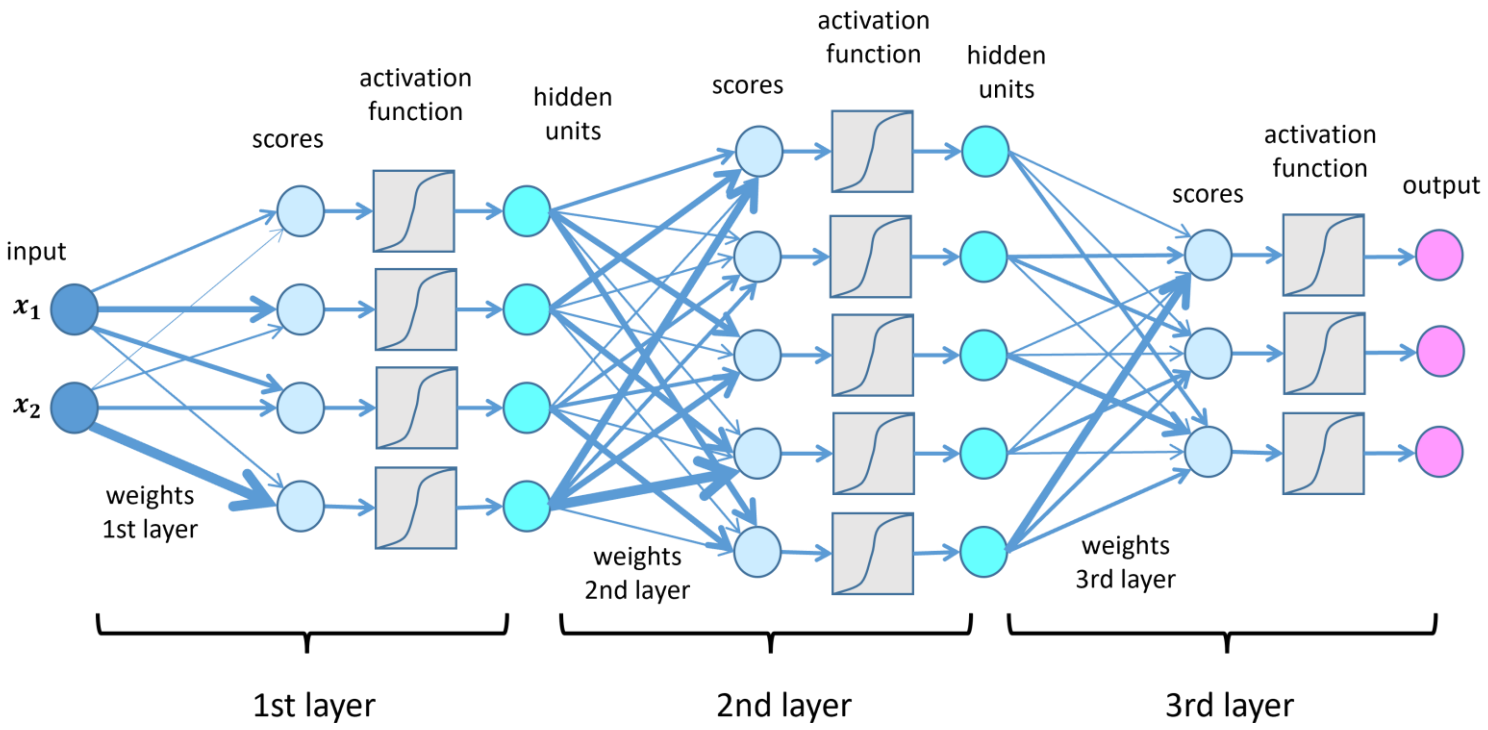


Figure 7.3 The tanh and ReLU activation functions.

https://www.geeksforgeeks.org/neural-networks-a-beginners-guide/

Deep Neural Networks (Deep Learning)

- **Multi-layer Perceptrons (MLP)** (1980s): the development of **backpropagation**
- AI Winter (1990s)
- ML/DL models (2000s-2010s) -> **Attention** (2017) -> AI Boom (2020s) ***ChatGPT (2022 Nov)***



- Deep : **Multiple** layers (more computation, higher performance)
- **Forward** Propagation: Linear, Non-liner Transform function
- **Backpropagation**: Loss Calculation, Gradient Calculation, Weight Update
- **Iteration**

https://lamarr-institute.org/blog/deep-neural-networks/

NN Language Models

1. How does a neural network language model (feedforward or recurrent) handle a large vocabulary, and how does it deal with sparsity (i.e. unseen sequences of words)?

Language Modeling: Calculating the probability of the next word in a sequence given some history.

- We've seen N-gram based LMs

Neural Network Language Models: Basically, a language model that utilises Neural Network. It can be FFNN, RNN, CNN, Transformer, and etc.

- Neural Network LMs far outperform n-gram language models
- Simple Feed Forward LMs can do better job!

Text Classification – Word Representation

How do we represent the meaning of a word?

Recap week3

- ***Count-based word representation :*** *(e.g.) One-hot vectors, Bag of Words, Term Frequency-Inverse Document Frequency (TF-IDF)*
- ***Prediction(Neural)-based word representation :*** *(e.g.) Word2Vec, GloVe*

week5

Word Embedding

- *Representing words by their **context***
- *Learn to encode similarity in the vectors themselves*

…government debt problems turning into ***banking*** *crises as happened in 2009…*
…saying that Europe needs unified ***banking*** *regulation to replace the hodgepodge…*
…India has just given its ***banking*** *system a shot in the arm…*

These context words will represent ***banking***

banking = $\begin{pmatrix} 0.286 \\ 0.792 \\ -0.177 \\ -0.107 \\ 0.109 \\ -0.542 \\ 0.349 \\ 0.271 \end{pmatrix}$

Ref.: Stanford CS224N NLP with Deep Learning

NN Language Models

1. How does a neural network language model (feedforward or recurrent) handle a large vocabulary, and how does it deal with sparsity (i.e. unseen sequences of words)?

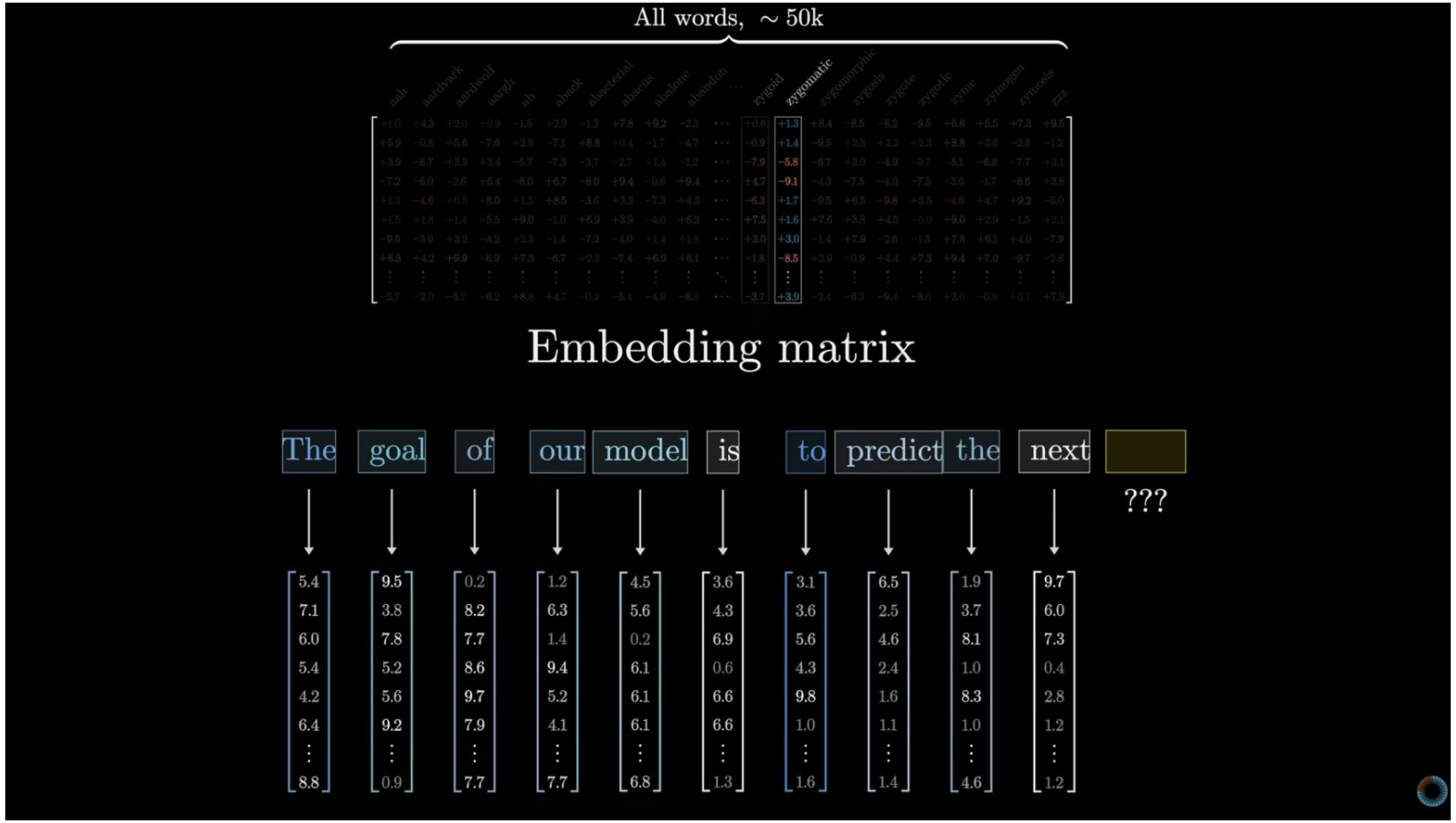
Word Embedding:

- NN LM projects words into a **continuous space** and represents each word as a low dimensional vector
- Capture **semantic and syntactic** relationships between words
- allowing the model to generalise better to **unseen sequences** of words.

(e.g.)

- **Seen:** *the cat is walking in the bedroom*
- **Unseen:** *a dog was running in a room*
    - **Neural Network LM can** use similarity of *(the, a), (dog, cat), (walking, running)* embeddings to generalise and predict the next word.
    - **Count-based N-gram LM would struggle,** as *(the, a)* or *(dog, cat)* are distinct word types from the N-gram LM's perspectives.

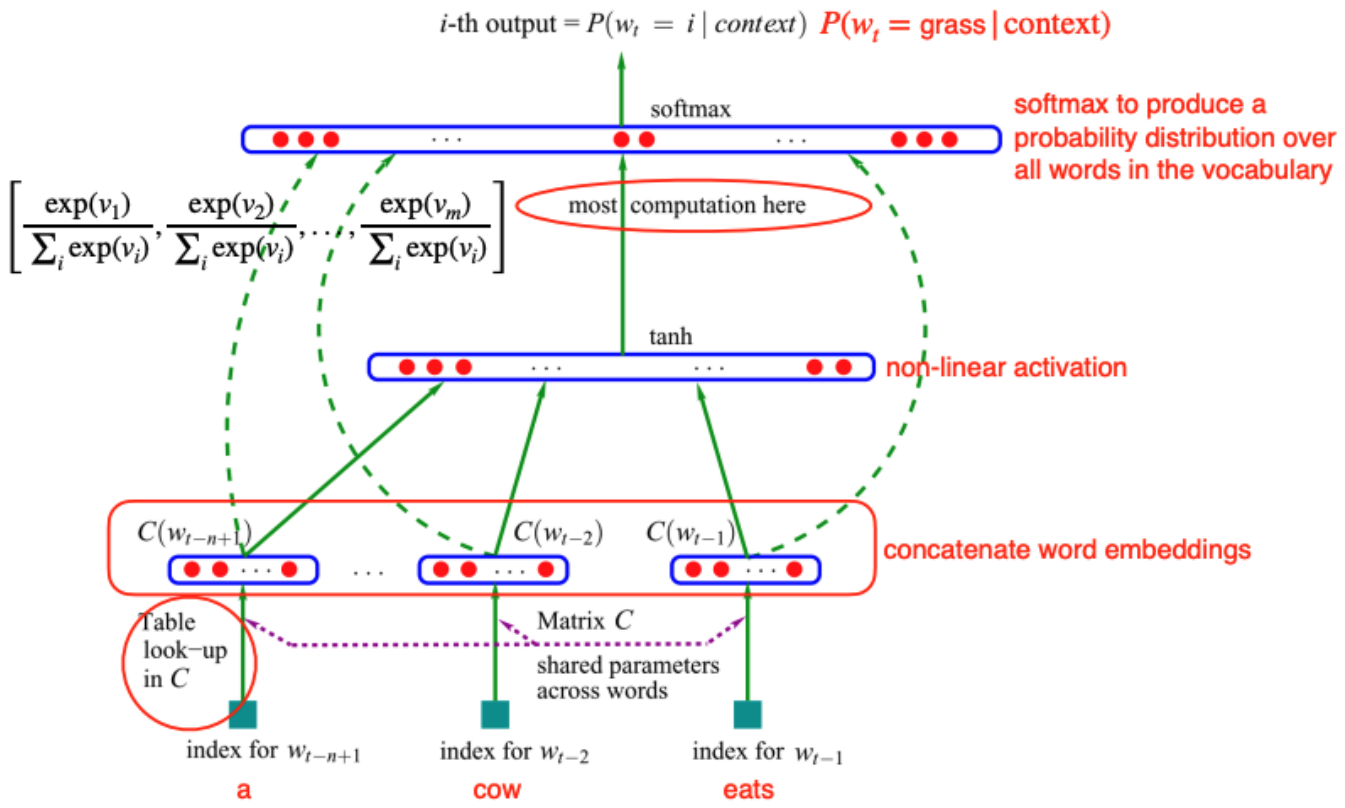
Word Embedding



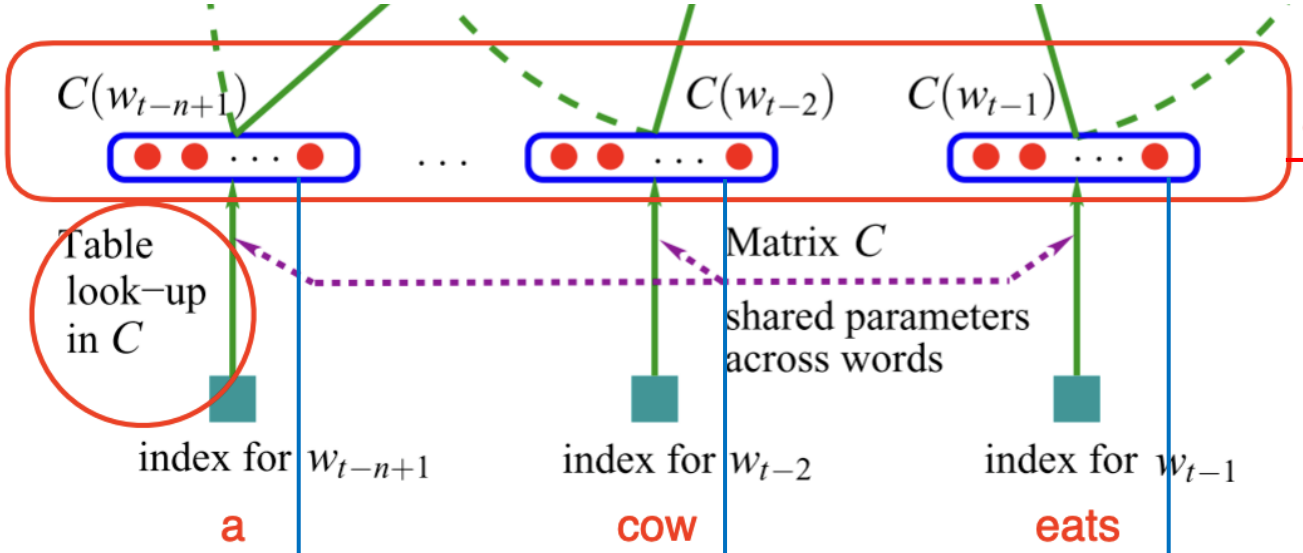
Transformers (how LLMs work) explained visually (by 3Blue1Brown):
https://www.youtube.com/watch?v=wjZofJX0v4M

Parameters of a NN Language Model

2. Why do we say most parameters of a neural network language model (feedforward or recurrent) is in their input and output word embeddings?



Ref.: Bengio et al, 2003 (NLP Lecture7 p25)



$d = 4$

$|V| = 5$

$N = 3$

(e.g.) Word Embedding Matrix (4-dim, Vocab = 5)

$|V|$

	a	grass	eats	hunts	cow
d	0.9	0.2	-3.3	-0.1	-0.5
	0.2	-2.3	0.6	-1.5	1.2
	-0.6	0.8	1.1	0.3	-2.4
	1.5	0.8	0.1	2.5	0.4

Word embeddings W_1 $d_1 \times |V|$

4×5

Lookup word embeddings

$\times$

a cow eats

1	0	0
0	0	0
0	0	1
0	0	0
0	1	0

5×1

(e.g.) Initialise words to one hot vector

$=$

a

0.9
0.2
-0.6
1.5

4×1

*[0.9*1+0.2*0+(-3.3)*0 +(-0.1)*0+(-0.5)*0]*

*[0.2*1+(-2.3)*0+(0.6)*0 +(-1.5)*0+(1.2)*0]*

*[-0.6*1+(0.8)*0+(1.1)*0 +(0.3)*0+(-2.4)*0]*

*[1.5*1+(0.8)*0+(0.1)*0 +(2.5)*0+(0.4)*0]*

$\oplus$

cow

-0.5
1.2
-2.4
0.4

4×1

$\oplus$

eats

-3.3
0.6
1.1
0.1

4×1

$=$

0.9
0.2
-0.6
...
1.1
0.1

12×1

Parameters of FFNN Language Model

Consider a Feedforward NN with:

- 1 embedding layer of **300-D (d)**
- 1 hidden layer of **300-D (d_h)**
- **10K** unique words **($|V|$)**
- Assume we use **3 previous words** as context

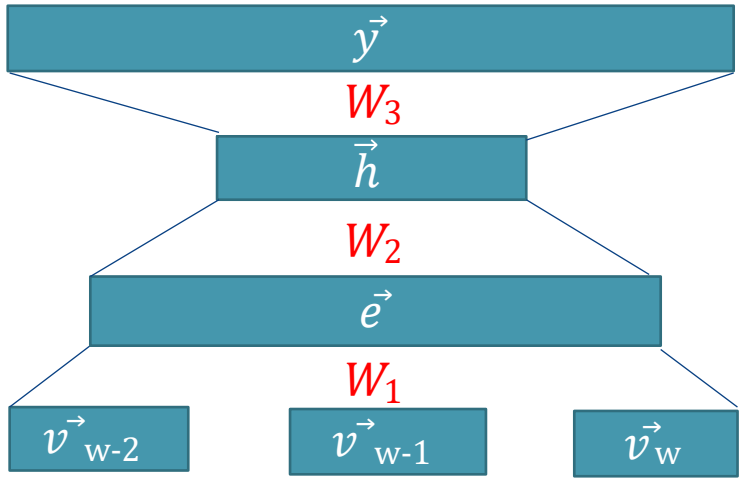
		# of parameters	
W_1	Input Embedding	$d \times |V|$	$300 \times 10K$
W_2	Weight	$d_h \times d \times 3$	300×900
b	Bias	d_h	300
W_3	Output Embedding	$|V| \times d_h$	$10K \times 300$

Output layer $\vec{y} = \text{softmax}(W_3\vec{h})$

Hidden layer $\vec{h} = \tanh(W_2\vec{e} + \vec{b})$

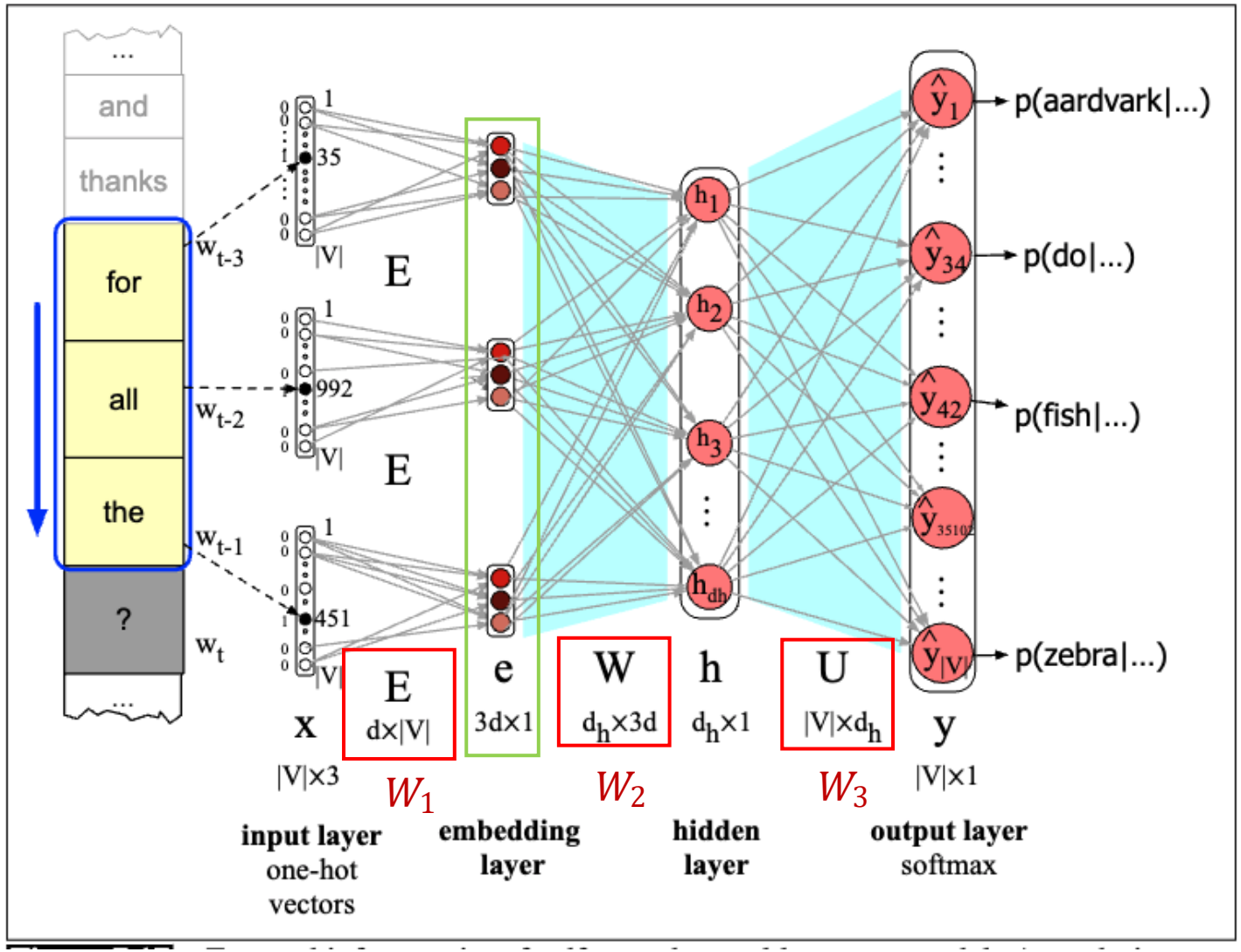
Embedding layer $\vec{e} = \vec{e}_{w-2} \oplus \vec{e}_{w-1} \oplus \vec{e}_{w}$

One-hot encoding $\vec{v}_{w-2}, \vec{v}_{w-1}, \vec{v}_{w}$



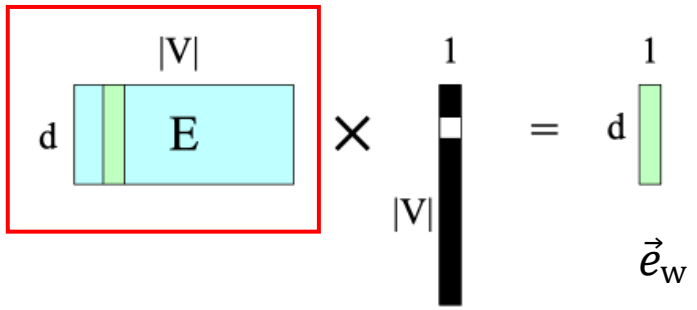
Note : Since 2026, the figure in JM3 has been updated to show the transposed version. This slide's image is remained as the 2025 version to align with the lecture.

FFNN Language Model



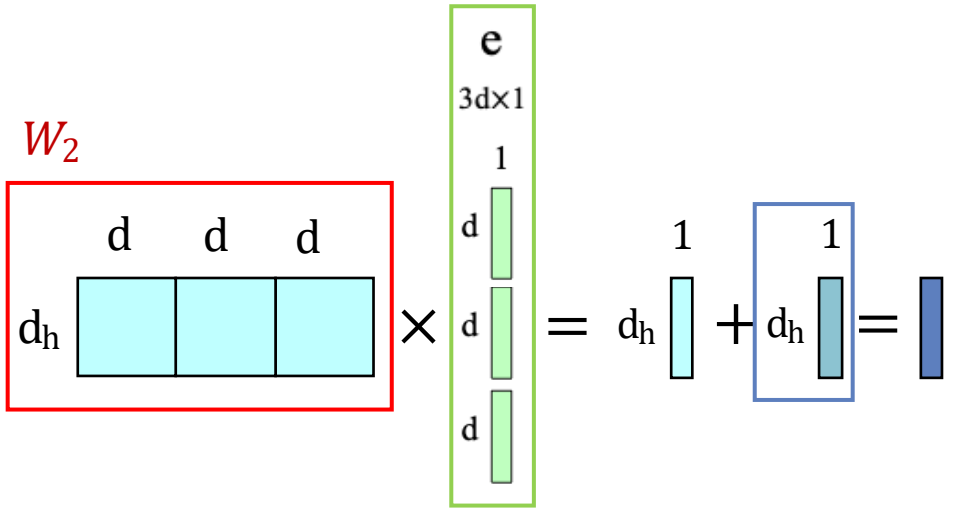
https://web.stanford.edu/~jurafsky/slp3/6.pdf

W_1

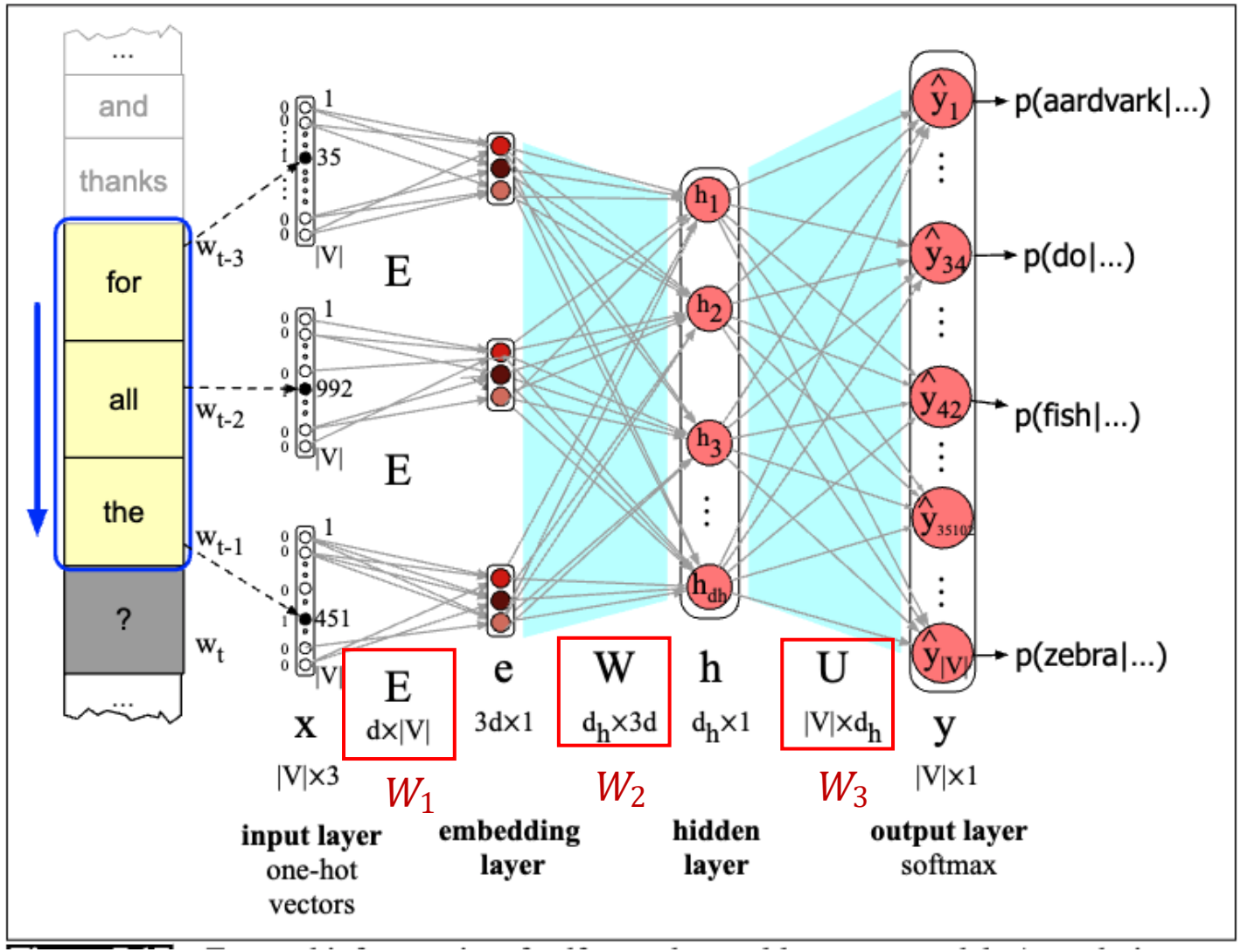


$$h = \tanh(W_2 \vec{e} + b)$$

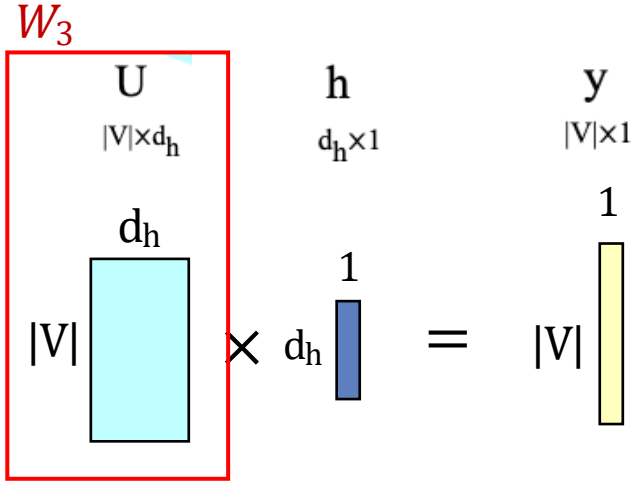
W_2



FFNN Language Model



$$\vec{y} = \text{softmax}(W_3 h)$$



https://web.stanford.edu/~jurafsky/slp3/7.pdf

Parameters of RNN Language Model

Consider a Recurrent NN with:

- 1 embedding layer of **300-D (d)**
- 1 hidden layer of **300-D (d)**
- **10K** unique words **($|V|$)**

		# of parameters	
Wx	Input Embedding	d× $\|V\|$	300 × 10K
W_h	Weight	d× d	300 × 300
B	Bias	d	300
Wy	Output Embedding	$\|V\|$ × d	10K×300

Output layer

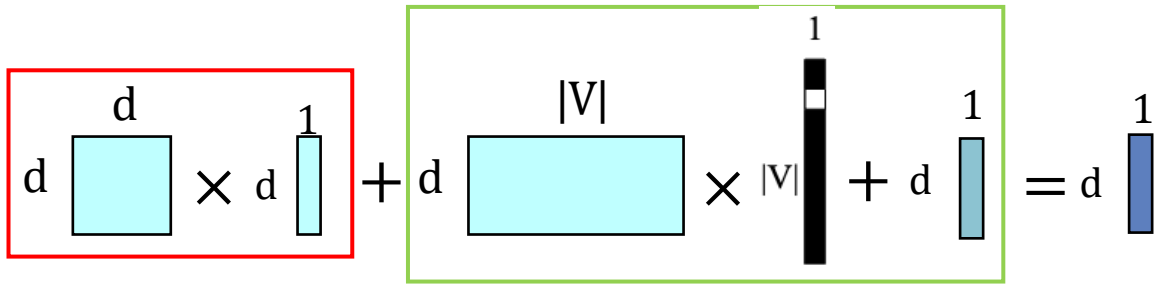
$$y_i = softmax(W_y h_i)$$

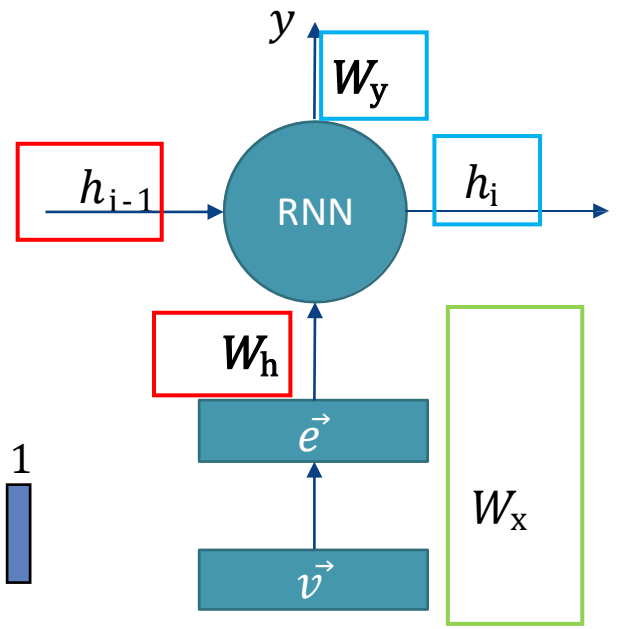
Hidden layer

$$h_i = \tanh(W_h h_{i-1} + W_x x_i + B)$$

Embedding layer

One-hot encoding





Advantage of RNN over N-gram?

3. What advantage does an RNN language model have over N-gram language model?

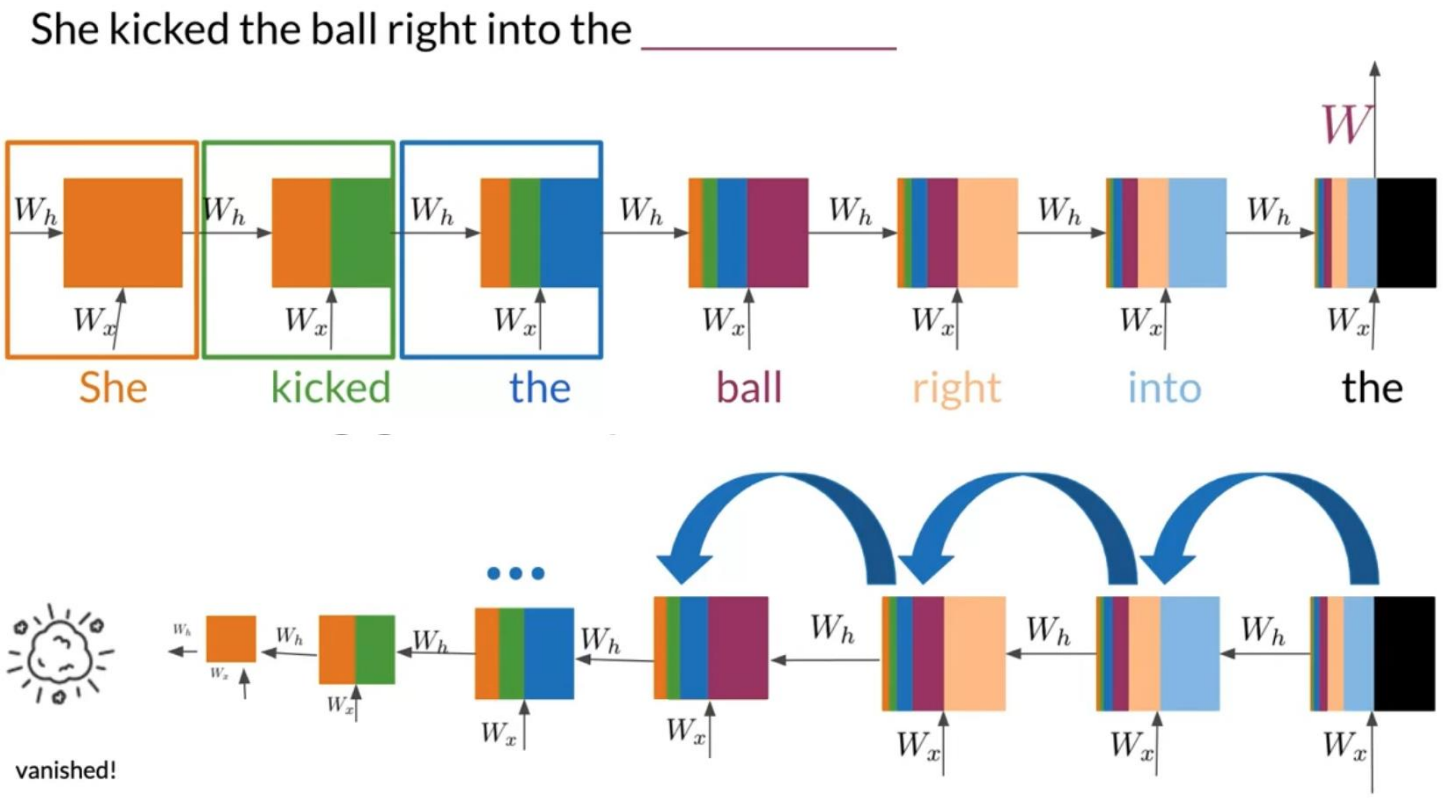
- **RNN language model** can capture **arbitrarily long contexts**, as opposed to an **N-gram language** model that uses a **fixed-width contexts**.

- RNN does so by **processing a sequence one word** at a time, applying a recurrence formula and using a state vector to represent words that have been previously processed.

- RNN generalises better to **unseen sequences** of words

Vanishing gradient in RNN

What is the vanishing gradient problem in RNN, and what causes it?



When we do **backpropagation** to compute the gradients, the gradients tend to get **smaller and smaller** as we move backward through the network.

Recurrent Neural Networks (RNNs) and Vanishing Gradients
https://www.youtube.com/watch?v=NgxMUHTJYmU

Vanishing gradient in RNN

What is the vanishing gradient problem in RNN, and what causes it?

- The net effect is that neurons in the **earlier layers learn very slowly**, as their gradients **are very small**. If the unrolled RNN is deep enough we might start seeing gradients **"vanish".**

- More sophisticated RNN models such as LSTM or GRU are introduced to tackle vanishing gradients. They do so by introducing "**memory cells**" that preserve gradients across time.

Programming!

Programming

1. In the iPython notebook `07-deep-learning`:

   - Can you find other word pairs that have low/high similarity? Try to look at more nouns and verbs, and see if you can find similarity values that are counter-intuitive.
   - We can give the neural models more learning capacity if we increase the dimension of word embeddings or hidden layer. Try it out and see if it gives a better performance. One thing that we need to be careful when we increase the number of model parameters is that it has a greater tendency to "overfit". We can tackle this by introducing dropout to the layers (`torch.nn.Dropout`), which essentially set random units to zero during training. Give this a try, and see if it helps reduce overfitting.
   - Improve the bag-of-words feed-forward model with more features, e.g. bag-of-N-grams, polarity words (based on a lexicon), occurrence of certain symbols (!).
   - Can you incorporate these additional features to a recurrent model? How?

Programming!

Dataset (Yelp Reviews)

- 1000 sentences, binary labels (positive, negative)
- Vocab size = 1812, including special tokens <unk> and <pad>

BoW NN (Model 1)

- 2 Linear layers
- BoW input features, hidden dimension=10
- Forward Network : relu - sigmoid

```
BowNetwork(
  (first_layer): Linear(in_features=1812, out_features=10, bias=True)
  (second_layer): Linear(in_features=10, out_features=1, bias=True)
```

Seq FFNN (Model 2)

- **Embedding layer** + 2 Linear layers
- Word embedding dimension=10, **input length(N)=30**, , hidden dimension=10
- Forward Network : embedding – **flatten** – relu - sigmoid

```
SequenceFFNetwork(
  (embedding): Embedding(1812, 10, padding_idx=1)
  (first_layer): Linear(in_features=300, out_features=10, bias=True)
  (second_layer): Linear(in_features=10, out_features=1, bias=True)
```

LSTM (Model 3)

- Embedding layer + **LSTM layer** + 1 Linear layer
- Word embedding dimension=10, hidden dimension=10
- Forward Network : embedding – init_hidden (**hidden state, cell state**) – **lstm** – hidden - sigmoid

```
SimpleLSTMNetwork(
  (embedding): Embedding(1812, 10, padding_idx=1)
  (lstm_layer): LSTM(10, 10, batch_first=True)
  (forward_layer): Linear(in_features=10, out_features=1, bias=True)
```